Scoring a Frozen Cycle Model: A Small Bit-Exact int8 CNN Accelerator for a RISC-V Microcontroller SoC, Its Compiler Backend, and a Pre-Registered Evaluation

Anirudh Sahu

Abstract*—We present MCUACCEL, a from-scratch int8 convolution, depthwise and fully connected accelerator memory-mapped into a PicoRV32 system-on-chip, together with a backend for the `mcugrad` compiler that emits chained command descriptors and driver C while every other operator stays in the compiler's own software kernels. The accelerator is bit-exact with the TFLite/gemmlowp integer reference. The evaluation is organised as a pre-registration: five claims with fixed thresholds, and an analytic port-bound cycle model of the accelerator, its driver and the fallback operators, were committed to git (tag `m1`, `6533b1e`) before any datapath or sequencer RTL existed, and the RTL was then scored against the frozen predictions. We give the full architecture, every equation of the model, the backend, the verification campaign and the scored outcome. Over 108 accelerator commands at five array widths ($P \in \{1, 2, 4, 8, 16\}$) the frozen model predicts every command within 12.4 % (median signed error 0.2 %) and each network's accelerator-plus-driver time within -0.9 % and 0.4 %; 6 commands exceed their pre-registered bias, and we identify the mechanism after the fact. At $P = 8$ the measured end-to-end speedups over the same core are 46.8× (`mnist_cnn`) and 350.0× (`cifar_mobilenet`); 5,000 random layers and whole-network runs show zero mismatching bytes. The LUT budget is refuted: 7,313 LUT4 against 6,000, and the integrated SoC closes timing at 35.8 MHz against 61.0 MHz without the accelerator. All cycle counts are cycle-accurate RTL simulation, not board wall-clock.

***Index Terms*—Hardware accelerators, convolutional neural networks, integer quantization, RISC-V, performance modeling, pre-registration, FPGA, bit-exact verification.**

I. Introduction

QUANTIZED convolutional networks cost tens of millions of cycles per image on a microcontroller-class core: on the scalar RV32IM core of this study, the two networks we evaluate take 11,383,383 and 69,169,620 cycles per inference in software. That a dedicated int8 accelerator beats such a core by an order of magnitude or more is well established [1]–[4], and reproducing that effect is not our contribution. We ask two narrower questions. First, how well does a simple analytic cycle model, written down *before* the hardware exists, predict the cycle counts of a small memory-bound accelerator, with error tolerances fixed in advance? Second, when the accelerated operators are fast, which parts of a whole-network inference remain, and how do they change with the width of the multiply-accumulate array?

A. Sahu is an independent researcher (e-mail: anirudh12sahu@gmail.com).

Analytic models of accelerators are common [5]–[7], but they are usually developed alongside the design they describe, so their reported agreement includes whatever adjustment happened during development. We borrow the practice of pre-registration from the empirical sciences: the claims, their thresholds and the model's predictions were committed to version control before any datapath or sequencer register-transfer-level (RTL) code, with a written list of the model's known biases, and the RTL was scored at its first full measurement. Deviations found afterwards are logged and explained, not absorbed.

A. Pre-registered claims

The claims below are quoted in substance from `docs/SPEC.md`; the predictions are frozen at tag `m1` (commit `6533b1e`, `m1_predictions.csv` sha256 prefix `fd94f39a1a913ee6`).

- C1 The end-to-end speedup at $P = 8$ output channels per tile is at least 10× on both networks, with fallback layers and driver included. **C1b**: at least 100× on `cifar_mobilenet`.
- C2 Bit-exactness: zero mismatching output bytes on 1,000 random layers (C2a), in every intermediate tensor of both networks for 10 images (C2b), and in the final outputs of 100 images per network on the full SoC, against the integer simulator and against the software baseline's outputs (C2c).
- C3 At $P = 8$ the accelerator uses at most 6,000 LUT4, 40 `MULT18X18D` and 12 `DP16KD` on a Lattice ECP5 LFE5U-85F.
- C4 The frozen model predicts every accelerator command within 20 % at every P (C4a) and each network's accelerator-plus-driver time within 10 % at $P = 8$ (C4b).
- C5 The saturation width of the P sweep is reported, predicted versus measured, with its mechanism stated in advance; the falsifiable part is C4a at every P.

B. Contributions

1) A from-scratch int8 accelerator for convolution, depthwise and fully connected layers with a 32-bit private

memory port, chained in-memory descriptors and a single pipelined requantizer that implements the gemmlowp rounding rule bit-exactly (Sec. IV).
2) A compiler backend for `mcugrad` [8] that packs weights and parameters, splits layers that exceed the buffers, builds descriptor chains and emits a drop-in `model.c` with driver code, keeping every unsupported operator as the compiler's own kernel (Sec. V).
3) A closed-form cycle model of the accelerator, the driver and the whole network, with three architecture variants, frozen before the RTL together with its known biases (Sec. VI).
4) A measurement protocol that logs post-freeze changes instead of absorbing them, including a build-fairness fix that moved the software baseline after the predictions were frozen (Sec. VII), and a verification campaign at every array width (Sec. VIII).
5) The scored outcome: one claim refuted, one descriptive, the others confirmed, the post-hoc root causes of the largest prediction errors, the array-width ablation with measured Amdahl shares, and the synthesis results with their timing consequence (Sec. IX).

Everything in this paper, including every number, regenerates from the repository (Sec. XII). Compared with our six-page conference version, this article adds the full microarchitecture, the complete cost model including the alternative architectures, the backend and driver, the verification methodology, the per-width error and Amdahl analyses, the synthesis sweep and a threats-to-validity discussion.

II. Background and Related Work

A. Integer-only inference arithmetic

The numerical contract is the integer-only scheme of Jacob et al. [9]: real values are $S(q - Z)$ with scale S and zero point Z, products are accumulated in 32-bit integers, and the combined rescale is applied as a fixed-point multiplication followed by a rounding shift. gemmlowp [10] documents the scheme and its `fixedpoint.h` [11] defines the two primitives that make an implementation bit-exact: the saturating rounding doubling high multiply (SRDHM) and the rounding divide by a power of two (RDBPOT). The LiteRT quantization specification [12] fixes the data formats (symmetric per-channel int8 weights, asymmetric per-tensor int8 activations, int32 bias) but not the rounding, so "bit-exact with TFLite" has to point at code. Our specification is `mcugrad`'s integer simulator [8], whose C header restates SRDHM and RDBPOT and whose generated C is differentially tested against it.

B. Software inference on microcontrollers

TensorFlow Lite Micro [13] interprets models with reference or optimized kernels, CMSIS-NN [14] supplies SIMD kernels for Cortex-M (with a shift-only rescale that is not the gemmlowp arithmetic), and MCUNet [15] co-designs the network and an inference library. These works show that the kernels dominate inference time and that the choice of software baseline changes any speedup by a large factor; we therefore fix the baseline to the code the same compiler emits for the same core (Sec. VII).

C. Accelerators and their integration

Eyeriss [16], [17] introduced the weight-, output- and row-stationary taxonomy surveyed by Sze et al. [18]; the TPU [19] and NVDLA [20] are large weight-stationary and pipeline designs whose documentation states that memory bandwidth, not multiplier count, bounds utilisation. At SoC scale, Gemmini [1] attaches a systolic array to Rocket [21] through the RoCC coprocessor interface, VTA [3] drives a GEMM core with a two-level instruction set from a compiler stack, ESP [22] provides a tile-based platform with generated drivers, and SNAX [23] couples several accelerators to a RISC-V cluster with streamers into banked memory. Gemmini in particular reports that a depthwise-heavy MobileNet gains far less than a dense network, the regime of our `cifar_mobilenet`.

At microcontroller scale the dominant alternative is to extend the core. CFU Playground [2] grows a custom-instruction unit attached to VexRiscv, PULP-NN [24] and XpulpNN [25] use SIMD dot-product and MAC-and-load instructions on multicore clusters, Vega [26] combines such cores with a convolution engine, and DIANA [27] pairs a RISC-V host with digital and analog accelerators. Standard attachment points for such units include CV-X-IF [28] and PicoRV32's PCPI [29]. A depthwise-separable accelerator attached as a custom function unit has also been reported [4], as has a RISC-V cluster with a hardwired attention unit [30]. Our design takes the memory-mapped route instead: it needs no core or compiler change and the accelerator moves its own data, at the cost of configuration traffic and polling through the same memory. Dataflow FPGA designs such as FINN [31] and hls4ml [32] are the other end of the spectrum, with per-layer hardware and no host core in the loop. A Tiny Tapeout neural accelerator exists [33], but we found no published int8 CNN accelerator on that platform to compare with.

D. Analytic performance models

The roofline model [7] bounds performance by peak compute and memory bandwidth times operational intensity; our model is a cycle-level refinement of it with a single 32-bit word per cycle as the roof. Zhang et al. [5] used a roofline-based model to explore tilings of an FPGA accelerator and then built it; Qiu et al. [34] report the gap between convolution-only and whole-network throughput that we measure as an Amdahl share; Alwani et al. [6] pair an analytic model of fused-layer dataflow with a prototype. In all three the model and the design were developed together. Our model is not more sophisticated than theirs; what differs is that it was frozen, hashed and scored, with its known biases written down in advance.

E. Verification against software references

CFU Playground ships golden whole-inference tests and a software emulation of the custom unit [2]; NXP's Neutron runner validates converted models against golden kernels and lists fallback operators as unvalidated [35]; TensorLift lifts

TABLE I
POSITIONING (QUALITATIVE; FROM THE CITED SOURCES).

Work	attachment	reference	model
Gemmini [1]	RoCC (Rocket)	own SW	–
VTA [3]	instr. stream, DMA	TVM	–
CFU Pl. [2]	custom instr.	TFLM golden	–
PULP-NN [24]	ISA ext., cluster	–	–
Zhang [5]	FPGA standalone	–	co-developed
Alwani [6]	FPGA standalone	–	co-developed
This work	memory-mapped	compiler sim.	frozen first

accelerator RTL to tensor semantics and proves the arithmetic core equivalent with an SMT solver [36]; and an int8 transformer accelerator reports byte comparisons against a bit-exact golden model under UVM [37]. We test at three levels (random single layers, every intermediate tensor of both networks, and final outputs on the full SoC) against an independently fuzzed compiler simulator, with the fallback layers in the same network running the same code as the baseline.

III. System Overview

A. SoC

The host is PicoRV32 [29] in a fast configuration (single-cycle multiplier, barrel shifter, dual-port register file, 64-bit cycle counters, no compressed instructions, no interrupts) behind an adapter to the request/grant/response bus of corv32 [38], whose core was not yet functional and was therefore substituted (decision D1). An interconnect decodes the CPU's address to a sim console (character output, test result, 64-bit cycle counter), the accelerator's register file, and port A of a true dual-port on-chip RAM (1 MiB in simulation). The accelerator is the master on port B, a private 32-bit port with one word per cycle and one outstanding request, so CPU and accelerator never arbitrate (Fig. 1). There is no boot ROM; RAM is preloaded and the core starts at the RAM base. All cycle counts in this paper come from Verilator simulation of this SoC; the FPGA is used only for synthesis results.

B. Bus protocol

Every port speaks one simplified OBI-style protocol, restated from corv32's design document. A request is accepted at the end of a cycle with `req` and `gnt` both high; `gnt` may depend combinationally on the request. The master holds `req` and its payload stable until acceptance and may raise a new request only if no accepted request awaits its response or that response arrives in the same cycle, so a one-cycle-latency slave sustains one transfer per cycle with at most one outstanding request. Each accepted request receives exactly one response (`rvalid` for one cycle, with `rdata` and `err`) in a later cycle, in order, with no back-pressure; writes are acknowledged the same way. An error response means the access had no effect. A write is visible to any request accepted in or after the cycle of its response. All slaves in the SoC are zero-wait ($\texttt{gnt} = 1$, response one cycle after acceptance), but every master is tested against randomly stalling slaves, and a bus checker asserts the protocol rules on every port in every test (Sec. VIII).

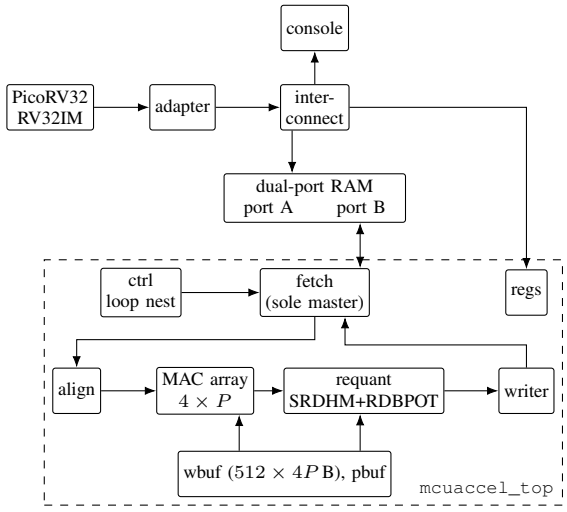


Fig. 1. System and accelerator block diagram (schematic; `docs/DESIGN.md` Sec. 1). The CPU writes a descriptor address and a start bit, then polls; the accelerator reaches RAM only through its own port.

C. Workloads

The workloads are `mcugrad`'s two int8 per-channel models, exactly the graphs that produced the compiler's committed generated code. `mnist_cnn` has two 3×3 convolutions (one and eight input channels), two max-pools, two transposes between NCHW and NHWC layouts and a fully connected layer. `cifar_mobilenet` is a MobileNet-style network: a 3×3 convolution on three channels, six depthwise 3×3 layers alternating with six 1×1 convolutions, an average pool and a fully connected layer. At $P = 8$ the backend maps them to 3 commands in 3 chains and 14 commands in 2 chains, respectively. In the software baseline's profile, the layers the accelerator can take account for 98.0 % (mnist) and 99.9 % (cifar) of the cycles.

IV. Accelerator Architecture

A. Dataflow

The datapath is output-stationary over one output pixel and a tile of P output channels. For each pixel in raster order and each tile, the reduction streams activation words from RAM and weight rows from a local buffer; P accumulators hold the tile's outputs, which are then requantized and written while the next tile runs.

Row runs. In NHWC layout, the in-bounds part of one kernel row of one output pixel, $x[i_y, i_{x0}{+}k_{x0}..i_{x0}{+}k_{x1}{-}1, 0..C{-}1]$, is a contiguous run of bytes. The full kernel row has $\mathrm{RUN} = K_W C$ bytes, consumed as $R_Q = \lceil \mathrm{RUN}/4 \rceil$ quads; run byte b is the tap $(b \operatorname{div} C, b \bmod C)$. Each cycle the MAC array takes one quad of activations (4 lanes) and one weight-buffer row of $4P$ bytes (word p holding output channel p's weights for the same four run bytes) and performs $4P$ multiply-accumulates. One mechanism therefore handles every channel count: a single-channel 3×3 layer costs one quad per kernel row rather than three taps, and three channels cost no more than

four. Runs that do not start on a word boundary pass through an aligner at the cost of one priming read per kernel row. At the borders, rows outside the input are skipped, columns are clipped to the quads that contain in-bounds bytes, and lanes outside the in-bounds byte range are masked so that their $(x - z_x)$ is forced to zero, which is exactly the reference's skipped padded taps. A fully connected layer is a convolution with $H = 1$, $W = N$, $K_H = K_W = 1$, $C = K$ and $O = M$.

Depthwise mode. For a depthwise layer (groups $= C = O$, C a multiple of four, aligned input), a tile is four channels; for each in-bounds tap one aligned activation word (four channels of one pixel) meets one weight word, and lane l feeds only accumulator l, using the multipliers of output group 0. Only 4 of the $4P$ multipliers do useful work, so depthwise layers are fetch-bound by construction; reusing activations across taps would need line buffers, which we modelled but did not build (Sec. VI-E).

Bandwidth. Everything goes through one 32-bit port. Convolution mode reaches $4P$ multiply-accumulates per activation word, so it is port-bound when a tile's reduction is long; descriptors, weights and parameters (once per command) and output writes also consume port cycles, writes with priority over reads. The single requantizer emits one output per cycle, so a tile with fewer activation words than outputs is requantizer-bound.

B. Software interface

Registers. The register file is a zero-wait slave with eight word registers: an identifier, a read-only configuration word (P, lanes, buffer rows), a control register (start, clear), a status register (busy, sticky done, sticky error), the first descriptor's address, a cycle counter of the last chain, a count of completed commands and the address of a faulting access. Start sets busy at the end of the accepting cycle, so every later status read sees it until the chain ends.

Descriptors. A command is a 18-word descriptor in RAM (Table II). Descriptors chain through their first word; the accelerator reads all words but the last and writes the last, the command's cycle stamp. Choosing in-memory chains over per-layer register writes (decision D8) means a chain of consecutive accelerated layers costs the CPU two stores and a poll, independent of its length; `cifar_mobilenet`'s thirteen consecutive accelerated layers form one chain.

Command execution. For each descriptor the sequencer reads the descriptor (DESC), loads the packed weight rows into the weight buffer (WLOAD) and three words per output channel into the parameter buffer (PLOAD), runs the loop nest (COMPUTE), waits until the MAC array, requantizer and writer are idle and no request is outstanding (FLUSH), writes the stamp (STAMP), and follows NEXT. The stamp counts the cycles from the first descriptor read up to the cycle before the stamp write, so per-command time is observable without a trace. On any error response the accelerator stops issuing, waits for the outstanding response, records the faulting address, resets its credits, empties the requantizer's queues, clears the writer and the MAC valids, and returns to idle with only the error bit set; a correct chain started afterwards must be bit-exact, which is tested.

TABLE II
COMMAND DESCRIPTOR (18 32-BIT WORDS; `DOCS/DESIGN.MD` SEC. 6.2).

Word	name	content
0	NEXT	next descriptor, 0 = last
1	CFG	mode (conv/fc or dw), K_H, K_W, S_H, S_W, P_H, P_W
2–3	IN_HW, OUT_HW	H, W; O_H, O_W
4	CH	C; NCH (output channels of this command)
5	RUN	RUN bytes $K_W C$; R_Q (0 in dw mode)
6	X_ORIGIN	x base $-P_H$ XROW $-$ P_W XPIX (mod 2^{32})
7–10	XPIX, XROW, steps	pixel, row and stride steps in bytes
11–12	Y_BASE, YPIX	first output address; output pixel step
13–14	W_BASE, WCFG	packed weights; rows to load, rows per tile
15	P_BASE	parameter records (bias, mult, shift)
16	QPARAM	z_x, z_y, act_min, act_max (int8)
17	STAMP	written by the accelerator: cycles of this command

C. Modules

Fetch. The only bus master. It arbitrates between the sequencer's read requests and the writer's (or stamp's) write requests, writes first; once a request is raised and not granted its selection is held, so the hold rule is met even if a write appears meanwhile. Every read carries a tag that returns with its response and routes it: descriptor word, weight-buffer row and word, parameter channel and field, or activation (with MAC enable, lane mask, alignment, first/last flags and weight row). Its idle output is combinational, so a wait phase ends in the cycle of the last response.

Sequencer (ctrl). A finite-state machine with idle, descriptor, weight-load and parameter-load phases (each with a wait state), a one-cycle pixel setup (PSETUP), a bubble-free issue state, flush, stamp and abort. Per pixel it computes the clipped kernel window and the run's first address with four 4-bit-by-32-bit shift-add products (no DSP), and in the issue state it raises a read every cycle except when the last request of a tile waits for a credit; row, tile and pixel changes cost no cycle because addresses are computed incrementally one cycle ahead. The timing in this paragraph is normative: it is the contract the RTL was built to and the model implements.

Credits. Two credits separate the sequencer from the requantizer, whose accumulator queue has depth two. Issuing a tile's last request takes a credit and pushes the tile's metadata (first channel, number of valid outputs NV, output address) to the requantizer; the requantizer returns the credit when it feeds the tile's last value into its pipeline. A post-review fix made the credit return combinational, matching the normative one-cycle loop (decision D20, Sec. VII).

Buffers. The weight buffer is P memories of 512 32-bit words read in parallel (one block RAM each), written word by word during WLOAD and read with the row carried in the activation's tag, so its output is valid in the response cycle. The parameter buffer holds bias, multiplier and shift for up to 512 channels in three memories.

Aligner. A register keeps the previous activation word; the quad is the current word when aligned, else the low 32 bits of

TABLE III
REQUANTIZER PIPELINE (ONE VALUE PER CYCLE).

Stage	operation
S0	read parameters of channel $ch_0 + i$; select accumulator i
S1	$s = \mathrm{wrap}_{32}(\mathrm{acc} + b)$; $a = \mathrm{wrap}_{32}(s \ll \ell_s)$
S2	64-bit product $a \cdot m$
S3	SRDHM, with the $-2^{31} \cdot -2^{31}$ special case
S4	RDBPOT by r_s (round half away from zero)
S5	33-bit $+z_y$, signed clamp, output byte

the concatenation of current and previous words shifted right by eight times the alignment. A priming read only loads the register.

MAC array. Stage 1 forms the 9-bit signed differences $(x - z_x)$, zeroed by the lane mask, and registers the $4P$ 17-bit products; stage 2 sums each output's four products and loads or adds into its 32-bit accumulator (convolution), or adds lane l's product into accumulator l (depthwise); stage 3 presents the accumulators with a valid flag at a tile's last quad. Accumulators wrap modulo 2^{32}; because the specification only uses the accumulator through $\mathrm{wrap}_{32}(\mathrm{acc} \ll \ell_s)$, a wrapping accumulator is exact for every input, and the backend additionally enforces the no-wrap bound $|b_o| + K \cdot 255 \cdot 128 \le 2^{31} - 1$.

Requantizer. One pipelined unit, one value per cycle, six stages (Table III). It implements

$$\mathrm{SRDHM}(a, m) = \left\lfloor (am + 2^{30})/2^{31} \right\rfloor, \tag{1}$$

$$\mathrm{RDBPOT}(x, e) = (x \gg e) + \big[\rho > (\mu \gg 1) + [x < 0]\big], \tag{2}$$

$$q = \mathrm{RDBPOT}\big(\mathrm{SRDHM}(s \ll \ell_s, m),\, r_s\big),$$
$$y = \min\big(\max(q + z_y,\, a_{\min}),\, a_{\max}\big),$$

where SRDHM saturates to $2^{31} - 1$ in the single case $a = m = -2^{31}$, $\mu = 2^e - 1$, $\rho = x \,\&\, \mu$, $s = \mathrm{wrap}_{32}(\mathrm{acc} + b)$, $\ell_s = \max(\mathrm{shift}, 0)$, $r_s = \max(-\mathrm{shift}, 0)$, $\gg$ arithmetic. The closed form of (1) equals gemmlowp's truncating-nudge form; the sum with z_y is formed in 33 bits and compared signed before the clamp, because a 32-bit sum could wrap where the specification clamps. The 32×32 multiply maps to four `MULT18X18D`.

Writer. It assembles consecutive output bytes of a tile into a word with byte enables and hands the word to a one-entry pending write when the word is full or the tile ends; it backpressures the requantizer, which then freezes all its stages.

Pipeline timing. Relative to the response cycle r of an activation read, the products are registered at the end of r, the accumulation at the end of $r + 1$, and at a tile's last quad the accumulators enter the requantizer's queue at the end of $r + 2$; with a response latency λ, issue to accumulator availability takes $\lambda + 3$ cycles. This latency, the two credits and the requantizer's rate are the three quantities the cost model needs besides the port.

V. BACKEND AND DRIVER

A. From compiler graph to commands

The backend (`backend/mcuaccel_backend`) consumes the int8 graph built by `mcugrad`'s own pipeline (trained weights, tracing, optimisation, calibration, per-channel quantisation) and checks that emitting it with `mcugrad`'s code generator reproduces the committed `model.c` byte for byte, so the accelerated and baseline builds start from the same model. A support check accepts convolutions with one group and kernel, stride and padding within the descriptor's fields (padding smaller than the kernel), depthwise convolutions with a channel count divisible by four, and matrix multiplications as 1×1 convolutions; everything else (pooling, transposes, padding, additions, softmax, and any convolution outside the envelope) falls back to the compiler's kernel, pasted byte for byte as the compiler emits it. Two optional passes (dropping an identity transpose, folding a transpose into fully connected weights) exist but are pre-registered as off in every scored build.

Packing. In convolution mode, for a command covering output channels $o_s..o_s{+}\mathrm{NCH}{-}1$ with $N_T = \lceil \mathrm{NCH}/P \rceil$ tiles, weight row $r = (tK_H + k_y)R_Q + j$ holds, in word p and byte l, the weight $w[o_s{+}tP{+}p][k_y][b \operatorname{div} C][b \bmod C]$ with $b = 4j + l$, or zero past the run or past NCH. A command loads $N_T K_H R_Q$ rows of P words. In depthwise mode, row $r = gK_HK_W + k_yK_W + k_x$ is one word holding the tap weights of channels $c_0{+}4g..c_0{+}4g{+}3$. Each output channel has a three-word parameter record: the graph's unfolded int32 bias, the multiplier and the shift, of which the hardware keeps the low six bits sign-extended. The accelerator computes the unfolded form, with the bias added in the requantizer; `mcugrad`'s folded bias would need per-pixel padded-tap corrections in hardware (decision D10).

Splitting. A command's weights must fit 512 buffer rows and its channels the parameter buffer. Convolutions are split over output channels into commands of $P\lfloor 512/(K_H R_Q) \rfloor$ channels that share the input and offset the output, weight and parameter bases; depthwise layers are split over channels into commands of $4\lfloor 512/(K_H K_W) \rfloor$; a layer whose single tile needs more rows than the buffer falls back to software. Because a command holds $P\lfloor 512/(K_H R_Q) \rfloor$ channels, splits occur in the two workloads only at small P: 32, 23 and 19 scored commands at $P = 1, 2, 4$ against 17 unsplit commands at $P = 8$ and 16.

Guards. The aligner and border clipping may read a few bytes outside a tensor; they are masked and never written. The backend cannot see where the network input will live, so this is enforced as a link-time property: the linker script pads both RAM edges and asserts that no section lies near them. The backend also forbids overlap of a command's input and output ranges.

B. Executor

A pure-Python executor of the descriptor semantics (`exec.py`) is the reference for the RTL's functional behaviour. Where the design text left room for interpretation, the executor fixes it (for example, the accumulator wraps per tile, the priming read happens only when misaligned, split descriptors are contiguous), and its docstring lists every such resolution; the RTL is required to match the executor, and the executor is tested equal to `mcugrad`'s integer operators

on seeded random layers and forced splits. Its optional trace (every activation read with address, mask, alignment and flags, and every tile's metadata) is the golden request stream for the sequencer's unit test, which checks addresses, tags and order but never cycle counts, so the RTL's timing is not tuned to the model through the tests.

C. Generated code and driver

The emitter writes a `model.c` with the same `model_run` interface as the compiler's: packed weights and parameters as constant arrays, one descriptor array per chain, and per-chain functions. All address fields are emitted as offsets; a bind step on the first call adds the real bases (next pointers, input, output, weight and parameter addresses), and each later call re-patches only the fields that depend on the network's input and output (the first layer's input origin and the last layer's output base). The emitted file records its P, and bind compares it with the accelerator's configuration register, so firmware built for one width never silently runs on another.

The driver (`sw/driver`) writes the descriptor address, writes start, polls the busy bit, and on error writes a failure code to the console and stops. Compiler barriers before the register writes and after the poll loop prevent cached arena values from being reused across a chain, and addresses are computed in `uintptr_t` rather than through out-of-object pointer arithmetic. The driver was written and measured on a stub accelerator before the model was frozen, so the model's driver constants are measurements: 71 cycles per chain call and 15 per patched field.

VI. Cost Model

The model (`model/m1.py`; normative text in `docs/DESIGN.md` Sec. 7) is a loop nest over the output pixels and channel tiles of a command with one closed form per tile. It was implemented from the design text, and a reviewer recomputed three layers by hand before it was frozen.

A. Per-pixel geometry

For output pixel (o_y, o_x) the sequencer computes, in its setup cycle,

$$\begin{aligned} i_{y0} &= o_y S_H - P_H, \qquad i_{x0} = o_x S_W - P_W, \\ k_{y0} &= \max(0, -i_{y0}), \qquad k_{y1} = \min(K_H,\, H - i_{y0}), \\ k_{x0} &= \max(0, -i_{x0}), \qquad k_{x1} = \min(K_W,\, W - i_{x0}), \\ b_0 &= k_{x0} C, \quad b_1 = k_{x1} C, \quad j_0 = \lfloor b_0/4 \rfloor, \quad j_1 = \lceil b_1/4 \rceil. \end{aligned} \tag{3}$$

In convolution mode a kernel row k_y whose run starts at address $A_{k_y} + 4j_0$ costs

$$R_{\mathrm{row}}(k_y) = (j_1 - j_0) + \big[(A_{k_y} + 4j_0) \bmod 4 \neq 0\big] \tag{4}$$

activation words (the bracket is the priming read), and the pixel needs $R = \sum_{k_y=k_{y0}}^{k_{y1}-1} R_{\mathrm{row}}(k_y)$ words per tile. In depthwise mode $R = (k_{y1} - k_{y0})(k_{x1} - k_{x0})$. Activations are re-read for every tile.

Tile t has $NV_t = \min(P, \mathrm{NCH} - tP)$ valid outputs in convolution mode and $NV_t = 4$ in depthwise mode, and its outputs need

$$W_t = \big\lceil \big((y_t \bmod 4) + NV_t\big)/4 \big\rceil \tag{5}$$

word writes at output address y_t. These writes are issued while later tiles are fetched; each steals one port slot from them.

B. Tile, pixel, command and chain

Let λ be the cycles per bus transaction ($\lambda = 1$ on the zero-wait RAM), $D = 18$ the descriptor length, W_{w} the weight words loaded ($N_T K_H R_Q P$ in convolution mode, $(\mathrm{NCH}/4) K_H K_W$ in depthwise mode) and NCH the command's output channels. Then

$$T_{\mathrm{tile}}(t) = \max\Big(\lambda(R + W_t),\ NV_t,\ \Big\lceil \tfrac{NV_t + \lambda + 3}{2} \Big\rceil\Big), \tag{6}$$

$$T_{\mathrm{pix}} = 1 + \textstyle\sum_t T_{\mathrm{tile}}(t), \tag{7}$$

$$\begin{aligned} T_{\mathrm{cmd}} = {} & \big[(D-1)\lambda + 1\big] + \big[\lambda W_{\mathrm{w}} + 1\big] + \big[3\lambda\,\mathrm{NCH} + 1\big] \\ & + \textstyle\sum_{\mathrm{pix}} T_{\mathrm{pix}} + \big[2\lambda + 10 + NV_{\mathrm{last}}\big], \end{aligned} \tag{8}$$

$$T_{\mathrm{chain}} = \textstyle\sum_{\mathrm{cmd}} (T_{\mathrm{cmd}} + \lambda + 1) + 2. \tag{9}$$

The three terms of (6) are the port bound (the tile's reads plus the previous tile's writes), the requantizer bound (one output per cycle), and the two-credit latency bound for tiles with few outputs: with two result slots and an issue-to-accumulator latency of $\lambda + 3$, two tiles must cover the latency plus the requantizer's drain. The 1 in (7) is the setup cycle. The brackets of (8) are the descriptor, weight and parameter loads (each with its one-cycle wait), the compute phase and the final drain (the constant is the requantizer's six-stage latency plus four cycles of hand-off); T_{cmd} is exactly what the accelerator writes as the stamp. Equation (9) adds the stamp write per command and the start and done cycles visible to the CPU.

C. Closed forms for the workloads

With aligned tensors, $\lambda = 1$ and $NV = P$ when $\mathrm{NCH} \geq P$:

- 1×1 convolution with C and O multiples of four and of P (`cifar` ops 2, 4, …, 12): no borders, $R = C/4$, $W = \lceil P/4 \rceil$, and $T_{\mathrm{compute}} = O_H O_W \big(1 + \frac{O}{P} \max(\frac{C}{4} + \lceil \frac{P}{4} \rceil,\, P,\, \lceil \frac{P+4}{2} \rceil)\big)$. The layer is port-bound when C exceeds about $3P$ and requantizer-bound below.
- 3×3 depthwise with padding one (`cifar` ops 1, 3, …, 11): $W = 1$, $NV = 4$, and $T_{\mathrm{compute}} = O_H O_W + \frac{C}{4} \sum_{\mathrm{pix}} \max(R + 1, 4)$, independent of P.
- 3×3 convolution with $C \in \{1, 3, 8\}$ (`mnist` ops 1, 3; `cifar` op 0): R from (4); eight channels are always aligned, one and three channels pay the priming word on three of four alignments.
- Fully connected with K inputs and $\lceil 10/P \rceil$ tiles: $T_{\mathrm{compute}} = 1 + \sum_t \max(K/4 + W_t, NV_t, \lceil (NV_t + 4)/2 \rceil)$, and WLOAD dominates.

These forms already state the mechanism of C5: in convolution mode the port term still falls as $1/P$ because activations are re-fetched per tile, so the sweep should flatten because of the depthwise layers (fixed four multiply-accumulates per cycle), the single requantizer and the weight load (roughly constant in P), not because the 32-bit port saturates.

D. Whole network

The predicted accelerated `model_run` is the sum over chains of $T_{\text{chain}} + t_{\text{drv}} + n_{\text{patch}}\, t_{\text{patch}}$, with the measured $t_{\text{drv}} = 71$ and $t_{\text{patch}} = 15$, plus the measured baseline cycles of every operator that stays in software, plus the 120-cycle bench window present in both builds. The software term is a measurement carried through to obtain a predicted speedup; claim C4b scores only the accelerator-plus-driver part, so the prediction cannot be right merely because its largest term was measured. At $P = 8$ the frozen prediction is 18,736 accelerator-plus-driver cycles for mnist and 135,876 for cifar, with 433,242 and 62,756 software cycles, i.e. software at 95.8 % and 31.6 % of the predicted run.

E. Alternative architectures (modelled, not built)

Before choosing the design, the model was evaluated for two alternatives with the same multipliers and port. *Line buffer* ("lb"): each in-bounds input row is streamed once per output row and tile into a K_H-row buffer; in depthwise mode the multipliers become four channels times up to P adjacent output pixels. *Accumulator memory* ("acc"): activations are fetched once per pixel and all tiles accumulate in a memory, $T_{\text{pix}} = 1 + \max(\lambda(R + \sum_t W_t),\ \sum_t \sum_{k_y}(j_1 - j_0),\ \sum_t NV_t)$. At $P = 8$ the frozen predictions of summed stamps per inference are 135,672 (built), 105,603 (lb) and 122,325 (acc) for cifar, and 18,496, 16,791 and 17,709 for mnist (Fig. 2). The output-stationary design was built because it needs no line buffers and handles every channel count with one mechanism (decision D9); the predicted gain of the line buffer comes almost entirely from the depthwise layers.

F. Registered biases

Before freezing, the design document listed where the model is known to be wrong and by how much (the same list is inside the frozen summary file):

- The setup cycle is hidden behind the requantizer where a layer is requantizer- or credit-bound; the model keeps it. Named commands were given a registered over-prediction of a few percent, all others less than 0.5 % (Table IX, last column).
- The drain term counts the last tile's writes and outputs in both the tile and the drain, over-predicting by $2 + W_{\text{last}}$ cycles per command.
- The wait phases are exact only if the fetch unit's idle signal is combinational, as specified.
- The driver constants omit the per-call bind check and the poll loop's phase wait, a few tens of cycles per network.
- The C4b measurement interval contains profiling instrumentation and call overhead that the prediction does not.
- The output alignment of the final layer is assumed zero.

Each item states a direction and a magnitude, so a measured error can be checked against it (Sec. IX-D).

VII. Methodology

A. Pre-registration protocol

Tag `m1` precedes in git history every commit that adds a datapath or sequencer to the accelerator RTL; only a register-file stub that sets done without computing existed before it. The model is not changed to match the RTL; post-hoc model fixes would be reported beside the frozen predictions, which remain the scored ones (none was made). The RTL is not tuned to the model either: it is scored as it stood at its first full stamp measurement, and every later change that alters a stamp is logged with before and after values. Scored predictions are the rows of `m1_predictions.csv` for the built architecture, 17 commands at $P = 8$ and 108 over the sweep, joined on network, operator, P and split index; a measured command without a predicted row, or vice versa, counts as a violation. The repository has no public remote yet, so the tag's commit and the file hash have not been timestamped externally **[MEASURE: public timestamp of m1, HARDWARE_TODO.md item 1]**.

B. Timing protocol

Inference time is the `rdcycle` interval around `model_run` on the SoC CPU, including software layers, descriptor patching, polling and driver cost, over 10 fixed test images and 3 timed runs after one warm-up (decision D17: the simulator is deterministic, so more runs add only wall time). We report the median over runs of the per-run median over images, with the minimum and maximum over images. The bench window inside the interval costs 120 cycles in both builds. Simulation runs at a nominal 50 MHz, which only converts cycles to time; cycles are the claimed unit.

Per-command time is the stamp. Per-network accelerator-plus-driver time for C4b is measured in one instrumented profile build per network as the `model_run` interval minus the bench window minus the sum of the fallback operators' `rdcycle` intervals, per image, median over the timed images. Stamps per command are collected in a separate build that prints them; the stamp is a hardware value, so it does not depend on the instrumentation.

C. Baseline and fairness

The baseline is `mcugrad`'s generated `model.c` with its own kernels, compiled with the same GCC flags (`-O2 -march=rv32im_zicsr`) and run on the same SoC with the accelerator present and idle, with the same images and the same bench harness. The accelerated build differs only in the operators the accelerator takes. Fairness requires the fallback operators to be identical machine code in both builds. Text identity was not enough: GCC inlined and register-allocated the kernels differently depending on which other operators a file contained, so the same operator cost a few cycles more or less in the two builds. Both builds therefore pin inlining at build time (decision D19): every kernel is `always_inline` into its one operator function and every operator function is `noinline`, so each fallback operator is compiled from its own text alone; the source text is unchanged and the

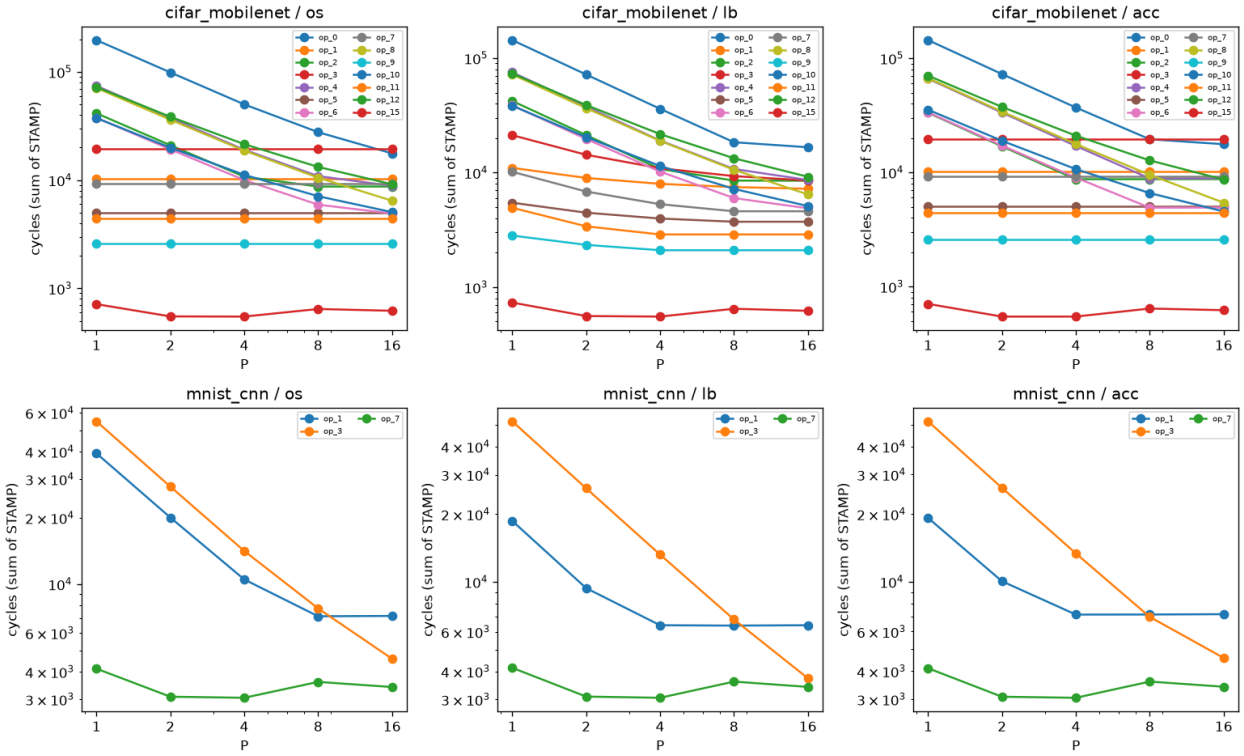


Fig. 2. Frozen per-command predictions (tag `m1`, unchanged since): predicted stamp cycles per command versus P for the built output-stationary design (os) and the two modelled alternatives (lb, acc). Flat lines are depthwise commands; the cifar fully connected command rises slightly at $P = 8$ because its weight load grows with the tile.

attributes are added by a build script. A second difference, a register move landing inside the first profiled operator because a bind call clobbered the argument register, was removed in the emitter. Identity is checked by measurement: in the two profile builds every fallback operator reports identical per-inference cycles. A shared non-inlined kernel file was also tried and rejected because it slowed both builds by removing constant propagation.

D. Deviations after the freeze

Two logged changes altered measured cycles; neither changed a frozen prediction or a threshold (values in `results/deviations.json`, written by a script that checks them against the decision log).

D19, the fairness fix. Pinning inlining moved the baseline from 12,188,028 to 11,383,383 cycles (mnist) and from 72,157,425 to 69,169,620 (cifar), and the accelerated $P = 8$ build from 451,480 to 243,059 and from 197,617 to 197,613; the measured speedup moved from 27.0× to 46.8× (mnist) and from 365.1× to 350.0× (cifar). No accelerator cycle changed. The mnist change is large because its accelerated runtime is dominated by pooling and transposes, which the earlier inlining compiled badly. Because the frozen speedup predictions used the pre-fix baselines and pre-fix fallback cycles, we add a post-hoc rescoring (Sec. IX-B).

D20, the credit loop. A red-team review of the RTL found a blocker (a credit returned during abort left the sequencer with three credits) and a credit return with one register stage more than the normative timing. The return was made combinational to match the design text written before the freeze; the model was not consulted. No full stamp measurement existed before the change, so the scored RTL is the fixed one. Smoke runs on the earlier RTL gave a C4b error of -1.0 % (mnist) and +0.4 % (cifar), against -0.9 % and +0.4 % on the scored RTL.

VIII. Verification

A. Unit level

Every accelerator and SoC module has cocotb tests on Verilator. A bus checker attached to every master and slave port in every test asserts the protocol rules each cycle (hold, one outstanding, response count and order, no response without request); the slave model is zero-wait or randomly stalling (random grant and response latency, seeded). The requantizer is tested against the specification's `requant` and clamp on directed edge cases (extreme accumulators, multipliers and shifts, the $-2^{31} \cdot -2^{31}$ case, equal clamp bounds, zero points that push the sum past 32 bits) and on random vectors, with back-pressure. The MAC array is tested in both modes for several P, the aligner on all alignments, the writer on random start alignments, tile sizes and acceptance delays, the fetch unit on random read/write mixes against stalling slaves, and the sequencer's request stream against the executor's trace.

B. Accelerator level (C2a)

A seeded random layer generator draws convolution, depthwise and fully connected layers with kernels, strides, padding,

spatial sizes and channel counts over the descriptor's envelope (including some layers with more than 512 output channels to force splits), full-range int8 activations and weights, random zero points, biases up to the no-wrap bound, multipliers in the normal and the full int32 range, shifts over the whole legal range, random clamps, input and output base alignments from zero to three bytes, and randomly stalling slaves. The backend packs each layer and builds its descriptors, the RTL runs them from a RAM model, and the output bytes are compared with the specification's layer functions. The campaign ran 1,000 layers at each width $P \in \{1, 2, 4, 8, 16\}$, 5,000 in total, with 0 mismatching bytes (C2a asks for 1,000 layers; the campaign ran that set at every width). It also ran 500 random descriptor chains, 1,450 runs with an error response injected mid-command (checking the error bit, the faulting address, the absence of further bus accesses, and bit-exactness of a following correct chain) and 10 start-while-busy checks, with 0 problems in total (RTL at commit `f9ef16f`).

C. Network level (C2b, C2c)

For C2b the int8 graph is run node by node for 10 images per network, accelerated nodes on the RTL through the backend's descriptors (30 and 140 accelerated node executions) and other nodes through the specification's operators, and every intermediate tensor is compared with the simulator: 0 mismatches. For C2c both firmware builds run on the full SoC for 100 test images per network and print their output bytes, which are compared with the integer simulator (0 mismatching bytes) and with the baseline firmware's outputs: 0 for mnist and 0 for cifar. C2c is therefore confirmed. The test sets are regenerated from the compiler's quantized images with 4-byte-aligned arrays, because the full test sets do not fit the simulated RAM.

D. Backend

The executor is tested equal to the specification on seeded random layers (`MCUACCEL_SEEDS` sets the count) and on forced splits; packing, descriptor fields and the worked example of the design document are tested directly. The emitted C is built with warnings as errors for the host and run against a host model of the accelerator, giving a host-side bit-exactness check of the generated code before any RTL is involved, and a textual test checks that every fallback kernel in the accelerated build is the baseline's text.

IX. Results

A. Baseline and Amdahl decomposition

Table IV gives the software baseline. The runs are deterministic: the three run medians are identical in every build, and the spread is across images only.

Table V decomposes both builds per operator at $P = 8$ from the profile builds. In the accelerated mnist run the chains account for 7.4 % of the cycles and the fallback operators (two max-pools and two transposes) for 92.6 %; in cifar the chains account for 68.2 % and the single average pool for 31.8 %. The fallback operators bound the speedup even for an accelerator that takes zero cycles: 50.6× for mnist and 1102.1× for cifar. Measured mnist is already close to its ceiling, so no accelerator change can move it much; the remaining work is in software.

TABLE IV
Software baseline on the SoC (cycles per inference over 10 images).

Network	median	min	max
`mnist_cnn`	11,383,383	11,378,852	11,387,883
`cifar_mobilenet`	69,169,620	69,144,241	69,186,835

TABLE V
Per-operator cycles per inference, baseline versus accelerated at $P = 8$ (profile builds, instrumentation included; means over the profiled inferences; cifar's chain includes the depthwise layers).

net	op (kind)	baseline	accel. P=8
mnist	0 (transpose)	23,344	23,344
mnist	1 (conv, chain)	3,352,201	6,437
mnist	2 (maxpool)	116,915	116,904
mnist	3 (conv, chain)	7,584,637	7,797
mnist	4 (maxpool)	60,748	60,753
mnist	5 (transpose)	23,952	23,952
mnist	7 (matmul, chain)	221,704	3,682
cifar	0–12 (conv, chain)	69,069,151	133,942
cifar	13 (avgpool)	62,764	62,764
cifar	15 (matmul, chain)	38,020	730

B. Speedup (C1, C1b)

Table VI compares the frozen prediction with measurement over the sweep. At $P = 8$ the measured speedups are 46.8× and 350.0×: C1 is confirmed and C1b is confirmed. The frozen predicted speedups (27.0× and 363.1×) were computed against the pre-D19 baselines (12,188,028 and 72,157,425 cycles) and pre-D19 fallback cycles, so they are not comparable one to one with the measured columns. The post-hoc row replaces only the software term: the frozen accelerator-plus-driver prediction, plus the measured fallback-operator cycles of the accelerated profile build (224,959 and 62,764), plus the bench window. It gives 243,815 and 198,760 cycles against measured 243,059 and 197,613, i.e. 46.7× and 348.0×. This row is post hoc by construction and is not scored.

C. Bit-exactness (C2)

Random layers: 5,000 over the widths $P \in \{1, 2, 4, 8, 16\}$, 0 mismatching bytes (C2a confirmed). Intermediate tensors: 10 images per network, 0 mismatches. Full SoC: 100 images per network, 0 mismatching bytes against the integer simulator, and 0 (mnist) and 0 (cifar) against the baseline firmware's outputs (C2c confirmed).

D. Prediction accuracy (C4)

Table VII summarises C4a per width and Table VIII lists every command at $P = 8$. Over all 108 scored rows the maximum absolute error is 12.4 %, the median signed error 0.2 %, and 0 rows exceed the 20 % tolerance: C4a is confirmed. Per network at $P = 8$, the measured accelerator-plus-driver

TABLE VI
END-TO-END SPEEDUP VERSUS P: FROZEN PREDICTION (AGAINST PRE-D19 BASELINES) AND MEASUREMENT; LAST ROW: POST-HOC RESCORING AT $P = 8$ IN THE PREDICTION COLUMNS.

	mnist		cifar	
P	pred.	meas.	pred.	meas.
1	22.9	35.2	111.7	107.2
2	25.2	41.2	187.1	179.3
4	26.4	45.0	282.0	271.2
8	27.0	46.8	363.1	350.0
16	27.2	47.5	411.4	398.4
8 post-hoc	46.7	46.8	348.0	350.0

TABLE VII
C4A PER WIDTH: SCORED COMMANDS, MAXIMUM ABSOLUTE AND MEDIAN SIGNED ERROR (%).

P	rows	max |err|	median err
1	32	0.6	0.02
2	23	0.9	0.02
4	19	1.4	0.20
8	17	12.4	0.31
16	17	12.4	0.34

time is 18,913 cycles (mnist) and 135,277 (cifar) against predicted 18,736 and 135,876, signed errors -0.9 % and 0.4 %, within the 10 % tolerance. Fig. 3 overlays the measurement on the frozen prediction.

The error is small and almost symmetric at $P \leq 4$ and grows at $P = 8$ and 16, where the requantizer term of (6) becomes comparable to the port term on narrow layers. The median signed error is positive at every width: the model over-predicts slightly, as the registered biases said it would.

Rows over the registered bias. 6 scored rows exceed their registered bias by more than 0.5 percentage points (Table IX); all are inside the tolerance, so C4a is unchanged, but the registered bias list was wrong for them. The root cause was found after measurement by probing the RTL (post hoc). Equation (7) adds the setup cycle to the tile terms. When the port term $\lambda(R + W)$ ties the requantizer term NV, the writer's pending write of the previous tile is granted during the setup cycle, which is therefore hidden; the RTL behaves as $T_{\text{pix}} = \max(1 + \lambda R,\ \lambda(R + W),\ NV,\ \dots)$ rather than $1 + \max(\dots)$. The registered bias counted only strictly requantizer-bound pixels. The tie is common in exactly the two layers that dominate the table. In `mnist` op 1 ($C = 1$, 3×3) at $P = 8$, an interior pixel needs one quad per kernel row plus a priming word whenever the row is misaligned, so $R = 6$ on misaligned pixels, $W = 2$ and $NV = 8$: $R + W = NV$. The probe found 546 of 784 setup cycles coinciding with a write grant. In `cifar` op 0 ($C = 3$) at $P = 16$, misaligned pixels have $R = 12$, $W = 4$ and $NV = 16$, again a tie; 690 of 1,024 setup cycles coincide with a write grant. The other four rows of the table were not probed; the tie is the likely cause. Separately, the measured drain after the last issue was one cycle longer than the design's constant, so the drain over-prediction is $1 + W_{\text{last}}$ rather than $2 + W_{\text{last}}$ cycles per command. The model outputs stay as scored (decision D21).

TABLE VIII
PER-COMMAND CYCLES AT $P = 8$ (PRED. FROM TAG `m1`; MEAS. = MEDIAN STAMP OVER THE TIMED IMAGES).

net	op	kind	K	pred.	meas.
mnist	1	conv	3x3	7,144	6,356
mnist	3	conv	3x3	7,756	7,715
mnist	7	fc	1x1	3,596	3,594
cifar	0	conv	3x3	27,724	27,029
cifar	1	dw	3x3	10,236	10,203
cifar	2	conv	1x1	8,712	8,452
cifar	3	dw	3x3	19,436	19,376
cifar	4	conv	1x1	10,888	10,883
cifar	5	dw	3x3	5,012	4,995
cifar	6	conv	1x1	5,928	5,923
cifar	7	dw	3x3	9,204	9,176
cifar	8	conv	1x1	10,536	10,469
cifar	9	dw	3x3	2,580	2,571
cifar	10	conv	1x1	7,096	7,077
cifar	11	dw	3x3	4,436	4,424
cifar	12	conv	1x1	13,240	13,237
cifar	15	fc	1x1	644	642

TABLE IX
COMMANDS WHOSE ERROR EXCEEDS THE PRE-REGISTERED BIAS (CYCLES; ERROR AND REGISTERED BIAS IN %).

net	op	P	pred.	meas.	err. %	reg. %
mnist	1	8	7,144	6,356	12.4	3.4
mnist	1	16	7,168	6,380	12.4	3.4
mnist	3	16	4,580	4,515	1.4	0.5
cifar	0	4	50,092	49,381	1.4	0.5
cifar	0	8	27,724	27,029	2.6	0.5
cifar	0	16	17,648	16,616	6.2	1.9

The chain identity. The frozen check that the chain cycle register equals $\sum(\text{STAMP} + \lambda + 1) + 2$ failed on every chain call: the measured identity is $+1$ on 275 of 275 calls, and $+2$ holds on 0. The cycle that accepts the start write is not counted as busy. The RTL and the stamps are correct; (9) with $+2$ includes that cycle and is the CPU-visible term, so C4b is affected by at most one cycle per chain call.

Independent stamp check. The stamp is self-reported by the sequencer. A bus-level count from the first descriptor request to the stamp request equals the self-reported stamp on 3 of 3 commands (6,356, 7,715, 16,616 cycles), the first such check; the design asked for at least three.

E. Array-width ablation and saturation (C5)

Table X gives summed stamps per inference over the sweep, predicted and measured. Saturation width P^* is the smallest P whose summed stamp is within ten percent of that at $P = 16$ ("16" means not reached on the grid). The predicted width is 16 (mnist) and 16 (cifar) and the measured width 16 and 16: on this doubling grid neither network saturates by the pre-registered rule, as predicted; at $P = 8$ the summed stamps are still 1.24 and 1.22 times those at $P = 16$. End to end the picture differs: applying the same rule to the measured `model_run` gives 4 for mnist (predicted 2) and 16 for cifar (predicted 16).

Table XI puts the accelerator's share of the measured run next to the run itself. In mnist the accelerator's share falls from

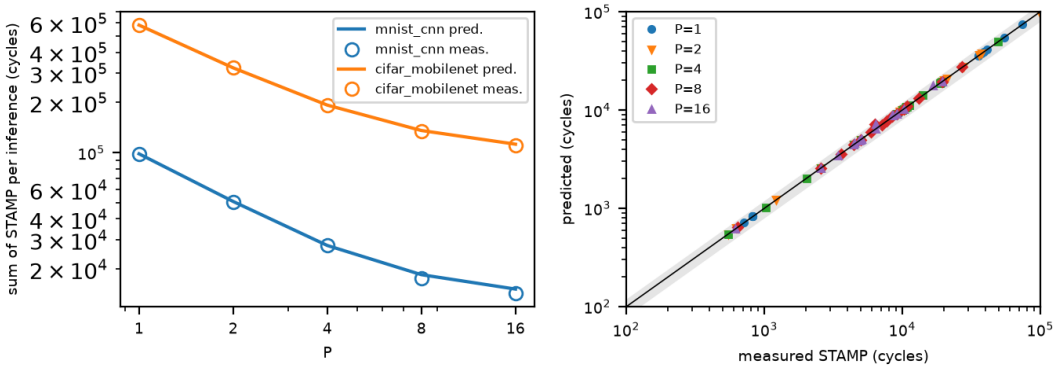


Fig. 3. Frozen prediction (tag `m1`, output-stationary architecture) against measurement. Left: summed stamps per inference versus P (line: prediction, circles: measurement). Right: every scored command, predicted versus measured cycles; the band is $\pm 20\,\%$.

TABLE X
SUMMED STAMPS PER INFERENCE (ACCELERATOR CYCLES ONLY) VERSUS P, FROZEN PREDICTION AND MEASUREMENT (MEDIAN OVER TIMED IMAGES).

	mnist		cifar	
P	pred.	meas.	pred.	meas.
1	98,312	98,240	582,676	581,949
2	50,888	50,707	322,660	322,472
4	27,697	27,626	192,788	191,898
8	18,496	17,665	135,672	134,457
16	15,156	14,299	112,308	110,490

TABLE XI
MEASURED `MODEL_RUN` CYCLES, SUMMED STAMPS, AND THE ACCELERATOR'S SHARE OF THE RUN (%) VERSUS P.

	mnist			cifar		
P	run	ΣSTAMP	acc. %	run	ΣSTAMP	acc. %
1	323,671	98,240	30.4	645,173	581,949	90.2
2	276,129	50,707	18.4	385,685	322,472	83.6
4	253,024	27,626	10.9	255,033	191,898	75.2
8	243,059	17,665	7.3	197,613	134,457	68.0
16	239,693	14,299	6.0	173,637	110,490	63.6

about a third at $P = 1$ to a few percent at $P = 16$, and the end-to-end run barely moves beyond $P = 4$: the pre-registered Amdahl reading. In cifar the accelerator still dominates at every width, the share falls steadily, and doubling P from 8 to 16 buys a further $350.0\times \rightarrow 398.4\times$. The mechanism is the one stated in advance (Fig. 2): the depthwise commands are flat in P, the 1×1 convolutions move from port-bound to requantizer-bound as P grows, and the weight load of the fully connected layers is roughly constant, while the 32-bit port itself never saturates in convolution mode because activations are re-fetched per tile.

F. Synthesis (C3) and its timing consequence

Table XII reports the Yosys/nextpnr-ecp5 flow on the LFE5U-85F [39] (out of context, the design inside a wrapper that registers every port, 50 MHz target). Resources are from the lowest seed of the newest RTL version of each design,

TABLE XII
SYNTHESIS ON ECP5 LFE5U-85F (RESOURCES FROM THE LOWEST SEED OF THE NEWEST RTL VERSION; FMAX MEDIAN OVER ITS DISTINCT SEEDS).

Design	LUT4	FF	MULT	BRAM	MHz	seeds
PicoRV32	2,896	1,027	4	0	81.7	10
Accel., $P = 1$	5,788	2,932	8	4	61.1	3
Accel., $P = 2$	5,922	2,998	12	5	60.4	3
Accel., $P = 4$	6,171	3,134	20	7	60.2	3
Accel., $P = 8$	7,313	3,542	36	11	61.5	10
Accel., $P = 16$	9,716	4,358	68	19	58.0	3
SoC, no accel.	3,392	1,073	4	128	61.0	10
SoC, accel. $P = 8$	10,684	4,427	40	139	35.8	10

Fmax the median over its distinct seeds. At $P = 8$ the accelerator uses 7,313 LUT4, 36 `MULT18X18D` and 11 `DP16KD`: the multiplier and block-RAM budgets hold, the LUT budget is exceeded by 21.9 %, so C3 is refuted. The budget followed a pre-RTL estimate of 3,800 LUT4 in the design document; we report the miss rather than revise the budget.

The estimate was wrong in its fixed part, not in its scaling. The multiplier and block-RAM counts follow the pre-RTL rules ($4P + 4$ multipliers: $4P$ in the array and four for the requantizer's 32×32 product; $P + 3$ block RAMs: P for the weight buffer and three for parameters) at every width in Table XII. The LUT count grows by about 262 per unit of P between $P = 1$ and $P = 16$, but already 5,788 LUT4 at $P = 1$: the fetch unit, the sequencer with its 18-word descriptor capture and loop counters, the requantizer with its queues, and the register file are a fixed cost that the multiplier array overtakes only at the largest widths. At $P = 8$ the accelerator is 2.5 times the LUT count of the PicoRV32 it serves.

The accelerator alone closes at 61.5 MHz at $P = 8$, above the 50 MHz target, and stays near that across widths. Integrated into the SoC, however, it closes at 35.8 MHz against 61.0 MHz without it in the same flow (a ratio of 0.59), and below the nominal 50 MHz of the cycle counts. If each SoC ran at its own median Fmax, the wall-time speedup would be the cycle speedup scaled by that ratio: an estimate of $27.5\times$ (mnist) and $205.3\times$ (cifar) at $P = 8$, still above both C1 thresholds. These

are estimates from out-of-context synthesis, not measurements; the board measurement is pending **[MEASURE: board wall-clock speedup, HARDWARE_TODO.md item 2]**. The SoC medians are over 10 and 10 placement seeds.

G. Summary of verdicts

C1 and C1b are confirmed and confirmed; C2a is confirmed, C2b found no mismatch, and C2c is confirmed; C3 is refuted; C4a is confirmed with 6 rows over their registered bias and C4b is within tolerance on both networks; C5 is descriptive, with predicted and measured saturation widths equal on the grid.

X. Discussion

A. What the frozen model got right and wrong

A page of closed forms, written from a design document before the RTL, predicted 108 commands with a median error of a fraction of a percent and the per-network accelerator time to within about one percent. We think three properties of the design made this possible, and they are also the conditions under which the result should be expected to transfer: the memory is a zero-wait private port, so λ is a constant; the sequencer's issue rule was written as a normative contract (one setup cycle per pixel, bubble-free issue, combinational idle), so the model and the RTL implement the same text; and the only shared resource is the port, whose arbitration (writes first) is simple. The model was wrong where two bounds tie: at a tie, an event the model charges serially (the setup cycle) overlaps with an event it charges in parallel (the previous tile's write). That is the class of error a fitted model would have absorbed into a constant; the frozen model makes it visible and attributable.

B. Design lessons

The 32-bit port, which the design was built around, is not what limits the accelerator at the widths studied. The single requantizer bounds the narrow 1×1 convolutions once P approaches the reduction length in words, the depthwise mode uses 4 of $4P$ multipliers by construction, and the weight load does not shrink with P. The frozen predictions for the modelled alternatives point to the next step: a line buffer for depthwise layers (Fig. 2); a second requantizer lane would address the narrow convolutions. For a network like `mnist_cnn`, no accelerator change matters: its accelerated run is dominated by pooling and layout operators in software, and the measured speedup is already close to the ceiling that a zero-cycle accelerator would reach. The build-fairness episode (D19) shows the same thing from the other side: a compiler inlining change moved mnist's measured speedup far more than any accelerator change in the sweep did.

XI. Threats to Validity

Construct validity. All cycle counts are Verilator simulation of the RTL at a nominal clock; there is no board measurement of wall time or power. Fmax comes from out-of-context synthesis, and the integrated SoC does not close at the nominal clock in this flow, so the cycle speedup overstates the wall-time speedup by the Fmax ratio (Sec. IX-F). The memory is zero-wait on-chip RAM; an external memory would add wait states and change the port-bound terms, although λ is the model's knob for that. Speedups are relative to one software baseline, the compiler's own C on a scalar core without SIMD; an optimized kernel library or an ISA extension would shrink them.

Internal validity. One person wrote the specification, the model and the RTL, so the independence between prediction and implementation rests on the git history (the tag precedes the datapath) rather than on separate authors; red-team reviews were performed by separate review passes but not by independent people. The model and the RTL implement the same normative timing text, so C4 measures how well the RTL meets its contract and how complete the closed forms are, not how well an outsider could predict an unknown design. The stamp is self-reported; it was checked against an independent bus-level count on 3 commands only. Two post-freeze changes altered cycles (D19, D20); both are logged with before and after values, and neither touched the predictions. Several result files were produced from a working tree with uncommitted changes; the files record this, the uncommitted changes did not touch the accelerator RTL (`rtl/accel` is identical at the verification-campaign commit, the timing-measurement commit and the release), and the SoC synthesis rows were rerun after the RAM's block-RAM mapping change, which left cycle counts unchanged.

External validity. Only two networks are evaluated, both from one compiler, both small. The P grid is a doubling grid, so the saturation width is coarse by construction. The design covers convolution, depthwise with multiplier one and fully connected layers only.

Conclusion validity. The simulator is deterministic: repeated runs give identical medians, so there is no run-to-run variance to report, only the spread across images. Claims are judged by fixed thresholds against single deterministic measurements; the remaining uncertainty is systematic, not statistical.

XII. Reproducibility

The repository contains the RTL, testbenches, backend, firmware, model, experiment scripts, raw results with commit, date and environment, and this paper. `scripts/bootstrap.sh` fetches pinned toolchains (an OSS CAD Suite release with Yosys, nextpnr-ecp5 and Verilator; an xPack RISC-V GCC; Python with cocotb) into a project-local directory; exact versions are in `docs/ENVIRONMENT.md`, and the compiler and the vendored CPU and bus are pinned by commit. `make test` runs the unit, accelerator and backend tests, `make lint` the Verilator and Yosys lint, and `make reproduce` runs the whole pipeline: SoC simulator build, baselines, driver timing, the verification campaign, the timed and profiled SoC runs at every P, the bit-exactness runs, the comparison with the frozen predictions, the synthesis sweep, the macro generation and the papers. Every number in this article is a macro generated by

`experiments/make_macros.py` from `results/`, or a table generated by the same script; the frozen predictions are read with `git show m1:`, never from the working tree. The full pipeline takes hours on a laptop, dominated by the cifar software baseline and the synthesis sweep; a quick mode runs a smoke version and restores the results afterwards.

XIII. Conclusion

We built a small int8 accelerator and its compiler backend and scored an analytic cycle model that was frozen before the RTL existed. The model held to within 12.4 % per command over five array widths and about one percent per network; the commands it missed most have an identified mechanism, a tie between two bounds that the model charged serially. The accelerator is bit-exact on every test run so far, the end-to-end speedups pass both pre-registered thresholds, and the LUT budget was missed, with a timing consequence for the integrated SoC that a board measurement still has to quantify. We hope the protocol, rather than the accelerator, is what transfers: freeze the model and its known biases, score the first measurement, log every later change, and let the residuals point at the mechanisms.

Acknowledgment

AI assistance (Anthropic's Claude) was used to draft code and text; the author takes responsibility for all content.

References

[1] H. Genc, S. Kim, A. Amid, A. Haj-Ali, V. Iyer, P. Prakash, J. Zhao, D. Grubb, H. Liew, H. Mao, A. Ou, C. Schmidt, S. Steffl, J. Wright, I. Stoica, J. Ragan-Kelley, K. Asanovic, B. Nikolic, and Y. S. Shao, "Gemmini: Enabling systematic deep-learning architecture evaluation via full-stack integration," in *58th ACM/IEEE Design Automation Conference (DAC)*, 2021, accessed 2026-10-10. [Online]. Available: https://arxiv.org/abs/1911.09925

[2] S. Prakash, T. Callahan, J. Bushagour, C. Banbury, A. V. Green, P. Warden, T. Ansell, and V. Janapa Reddi, "CFU Playground: Full-stack open-source framework for tiny machine learning (tinyML) acceleration on FPGAs," in *IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 2023, pp. 157–167, accessed 2026-10-10. [Online]. Available: https://arxiv.org/abs/2201.01863

[3] T. Moreau, T. Chen, L. Vega, J. Roesch, E. Yan, L. Zheng, J. Fromm, Z. Jiang, L. Ceze, C. Guestrin, and A. Krishnamurthy, "A hardware-software blueprint for flexible deep learning specialization," 2018, accessed 2026-10-10. [Online]. Available: https://arxiv.org/abs/1807.04188

[4] M. Yildirim and O. Ozturk, "RISC-V based TinyML accelerator for depthwise separable convolutions in edge AI," 2025, accessed 2026-10-10. Preprint, no peer-reviewed venue listed. [Online]. Available: https://arxiv.org/abs/2511.21232

[5] C. Zhang, P. Li, G. Sun, Y. Guan, B. Xiao, and J. Cong, "Optimizing FPGA-based accelerator design for deep convolutional neural networks," in *ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA)*, 2015, accessed 2026-10-10. [Online]. Available: https://www.microsoft.com/en-us/research/?p=595783

[6] M. Alwani, H. Chen, M. Ferdman, and P. Milder, "Fused-layer CNN accelerators," in *49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2016, accessed 2026-10-10. [Online]. Available: https://compas.cs.stonybrook.edu/~mferdman/downloads.php/MICRO16_Fused_Layer_CNN_Accelerators.pdf

[7] S. Williams, A. Waterman, and D. A. Patterson, "Roofline: An insightful visual performance model for multicore architectures," 2009, accessed 2026-10-10. [Online]. Available: https://zenodo.org/record/1236156

[8] Anirudh, "mcugrad: from-scratch deep learning framework and int8 C99 compiler for microcontrollers," Git submodule third_party/mcugrad at commit e9f6283, accessed 2026-10-10. [Online]. Available: https://github.com/anirudh12032008/mcugrad.git

[9] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," 2017, accessed 2026-10-10. [Online]. Available: https://arxiv.org/abs/1712.05877

[10] B. Jacob *et al.*, "gemmlowp: low-precision matrix multiplication, quantization doc," GitHub, google/gemmlowp, doc/quantization.md, accessed 2026-10-10. [Online]. Available: https://raw.githubusercontent.com/google/gemmlowp/master/doc/quantization.md

[11] ——, "gemmlowp: fixedpoint/fixedpoint.h (saturatingroundingdoublinghighmul, roundingdividebypot)," GitHub, google/gemmlowp, accessed 2026-10-10. [Online]. Available: https://raw.githubusercontent.com/google/gemmlowp/master/fixedpoint/fixedpoint.h

[12] Google LiteRT (TensorFlow Lite) team, "Litert 8-bit quantization specification," Google AI Edge documentation, accessed 2026-10-10. [Online]. Available: https://developers.google.com/edge/litert/performance/quantization_spec

[13] R. David, J. Duke, A. Jain, V. Janapa Reddi, N. Jeffries, J. Li, N. Kreeger, I. Nappier, M. Natraj, S. Regev, R. Rhodes, T. Wang, and P. Warden, "TensorFlow Lite Micro: Embedded machine learning on TinyML systems," in *Proceedings of the 4th MLSys Conference*, 2021, accessed 2026-10-10. [Online]. Available: https://arxiv.org/abs/2010.08678

[14] L. Lai, N. Suda, and V. Chandra, "CMSIS-NN: Efficient neural network kernels for Arm Cortex-M CPUs," 2018, accessed 2026-10-10. [Online]. Available: https://arxiv.org/abs/1801.06601

[15] J. Lin, W.-M. Chen, Y. Lin, J. Cohn, C. Gan, and S. Han, "MCUNet: Tiny deep learning on IoT devices," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020, accessed 2026-10-10. [Online]. Available: https://arxiv.org/abs/2007.10319

[16] Y.-H. Chen, J. Emer, and V. Sze, "Eyeriss: A spatial architecture for energy-efficient dataflow for convolutional neural networks," in *43rd ACM/IEEE International Symposium on Computer Architecture (ISCA)*, 2016, accessed 2026-10-10. [Online]. Available: https://dspace.mit.edu/handle/1721.1/102369

[17] Y.-H. Chen, T. Krishna, J. S. Emer, and V. Sze, "Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks," *IEEE Journal of Solid-State Circuits*, vol. 52, no. 1, 2017, accessed 2026-10-10. Accepted-manuscript PDF opened; volume and issue taken from the search listing, not from the PDF. [Online]. Available: https://courses.cs.washington.edu/courses/cse550/21au/papers/CSE550.Eyeriss.pdf

[18] V. Sze, Y.-H. Chen, T.-J. Yang, and J. Emer, "Efficient processing of deep neural networks: A tutorial and survey," 2017, accessed 2026-10-10. [Online]. Available: https://arxiv.org/abs/1703.09039

[19] N. P. Jouppi, C. Young, N. Patil, D. Patterson *et al.*, "In-datacenter performance analysis of a tensor processing unit," in *44th International Symposium on Computer Architecture (ISCA)*, 2017, accessed 2026-10-10. [Online]. Available: https://arxiv.org/abs/1704.04760

[20] NVIDIA, "NVDLA hardware architectural specification: Unit description (nvdlav1)," nvdla.org, accessed 2026-10-10. [Online]. Available: http://nvdla.org/hw/v1/ias/unit_description.html

[21] K. Asanović, R. Avizienis, J. Bachrach, S. Beamer, D. Biancolin, C. Celio, H. Cook *et al.*, "The rocket chip generator," EECS Department, University of California, Berkeley, Tech. Rep. UCB/EECS-2016-17, 2016, accessed 2026-10-10. [Online]. Available: https://www2.eecs.berkeley.edu/Pubs/TechRpts/2016/EECS-2016-17.html

[22] P. Mantovani, D. Giri, G. Di Guglielmo, L. Piccolboni, J. Zuckerman, E. G. Cota, M. Petracca, C. Pilato, and L. P. Carloni, "Agile SoC development with Open ESP," in *International Conference on Computer Aided Design (ICCAD), invited paper*, 2020, accessed 2026-10-10. [Online]. Available: https://arxiv.org/abs/2009.01178

[23] R. A. Antonio, J. Dumoulin, X. Yi, J. Van Delm, Y. Deng, G. Paim, and M. Verhelst, "An open-source HW-SW co-development framework enabling efficient multi-accelerator systems," in *ACM/IEEE International Symposium on Low Power Electronics and Design (ISLPED)*, 2025, accessed 2026-10-10. Venue taken from the arXiv comments field. [Online]. Available: https://arxiv.org/abs/2508.14582

[24] A. Garofalo, M. Rusci, F. Conti, D. Rossi, and L. Benini, "PULP-NN: Accelerating quantized neural networks on parallel ultra-low-power RISC-V processors," 2019, accessed 2026-10-10. [Online]. Available: https://arxiv.org/abs/1908.11263

[25] A. Garofalo, G. Tagliavini, F. Conti, L. Benini, and D. Rossi, "XpulpNN: Enabling energy efficient and flexible inference of quantized neural network on RISC-V based IoT end nodes," 2020, accessed 2026-10-10. Preprint; the arXiv page lists no venue. [Online]. Available: https://arxiv.org/abs/2011.14325

[26] D. Rossi, F. Conti, M. Eggimann, A. Di Mauro, G. Tagliavini, S. Mach, M. Guermandi, A. Pullini, I. Loi, J. Chen, E. Flamand,

and L. Benini, “Vega: A 10-core SoC for IoT end-nodes with DNN acceleration and cognitive wake-up from MRAM-based state-retentive sleep mode,” 2021, accessed 2026-10-10. [Online]. Available: https://arxiv.org/abs/2110.09101

[27] P. Houshmand, G. Sarda, V. Jain, K. Ueyoshi, I. Papistas, M. Shi, Q. Zheng, D. Bhattacharjee, A. Mallik, P. Debacker, D. Verkest, and M. Verhelst, “DIANA: An end-to-end hybrid DIgital and ANAlog neural network SoC for the edge,” *IEEE Journal of Solid-State Circuits*, 2023, accessed 2026-10-10. [Online]. Available: https://imec-publications.be/handle/20.500.12860/59550

[28] OpenHW Group, “CORE-V eXtension interface (CV-X-IF) specification, introduction,” OpenHW Group documentation, accessed 2026-10-10. [Online]. Available: https://docs.openhwgroup.org/projects/openhw-group-core-v-xif/en/latest/intro.html

[29] C. Wolf, “PicoRV32: a size-optimized RISC-V cpu, readme,” GitHub, YosysHQ/picorv32, accessed 2026-10-10. [Online]. Available: https://raw.githubusercontent.com/YosysHQ/picorv32/main/README.md

[30] P. Wiese, G. İslamoğlu, M. Scherer, L. Macan, V. J. B. Jung, A. Burrello, F. Conti, and L. Benini, “Toward attention-based TinyML: A heterogeneous accelerated architecture and automated deployment flow,” 2024, accessed 2026-10-10. Accepted to IEEE Design & Test per the arXiv page. [Online]. Available: https://arxiv.org/abs/2408.02473

[31] Y. Umuroglu, N. J. Fraser, G. Gambardella, M. Blott, P. Leong, M. Jahre, and K. Vissers, “FINN: A framework for fast, scalable binarized neural network inference,” in *25th International Symposium on Field-Programmable Gate Arrays (FPGA)*, 2017, accessed 2026-10-10. [Online]. Available: https://arxiv.org/abs/1612.07119

[32] J. Duarte, S. Han, P. Harris, S. Jindariani, E. Kreinar, B. Kreis, J. Ngadiuba, M. Pierini, R. Rivera, N. Tran, and Z. Wu, “Fast inference of deep neural networks in FPGAs for particle physics,” *Journal of Instrumentation*, vol. 13, p. P07027, 2018, accessed 2026-10-10. [Online]. Available: https://arxiv.org/abs/1804.06913

[33] G. Chadwick, “Tiny neural network accelerator (tt_um_gregac_tiny_nn), tiny tapeout IHP 25a, project 678,” Tiny Tapeout project page, accessed 2026-10-10. [Online]. Available: https://tinytapeout.com/chips/ttihp25a/678

[34] J. Qiu, J. Wang, S. Yao, K. Guo, B. Li, E. Zhou, J. Yu, T. Tang, N. Xu, S. Song, Y. Wang, and H. Yang, “Going deeper with embedded FPGA platform for convolutional neural network,” in *ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA)*, 2016, pp. 26–35, accessed 2026-10-10. [Online]. Available: https://hgpu.org/?p=15841

[35] NXP Semiconductors, “eIQ Neutron Runner: validating TFLite models against bit-exact golden kernels,” NXP eIQ documentation, accessed 2026-10-10. [Online]. Available: https://eiq.nxp.com/learning-hub/tools/neutronSdk/softwareTools/NeutronRunner.html

[36] R. Gao, H. Jin, J. Yang, and N. Bleier, “TensorLift: Automatic extraction of tensor-level ISA semantics from accelerator RTL via MLIR semantic lifting,” 2026, accessed 2026-10-10. Preprint (v3, 26 Aug 2026). [Online]. Available: https://arxiv.org/abs/2604.13523

[37] Shashank, “Transformer accelerator (TFA): A macro-op INT8 hardware chip for transformer inference and machine translation,” 2026, accessed 2026-10-10. Single-author preprint, not peer reviewed. [Online]. Available: https://arxiv.org/abs/2608.23582

[38] Anirudh, “corv32: five-stage RV32IM core and SoC (design docs, bus protocol),” Unpublished repository; vendored subset at third_party/corv32 of the mcuaccel repository, commit e1d0bc0, accessed 2026-10-10.

[39] Lattice Semiconductor, “ECP5 / ECP5-5G product page and family table,” latticesemi.com, accessed 2026-10-10. [Online]. Available: https://www.latticesemi.com/en/Products/FPGAandCPLD/ECP5